EEE 6778. Applied Machine Learning II

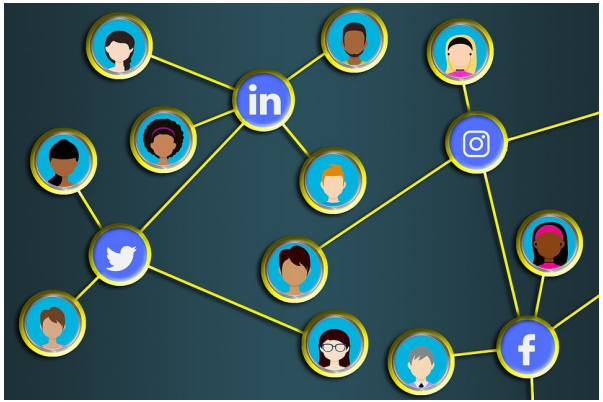
Module 6. Graph Neural Networks

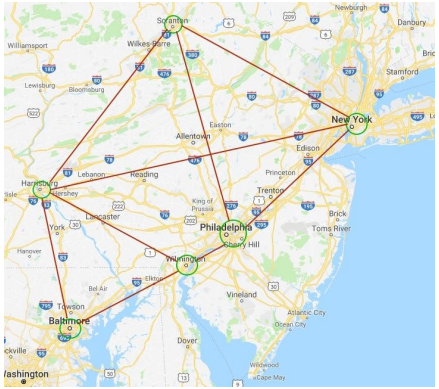
Why Graphs?
How GNNs work?
Types of GNNs
Applications
Implementation

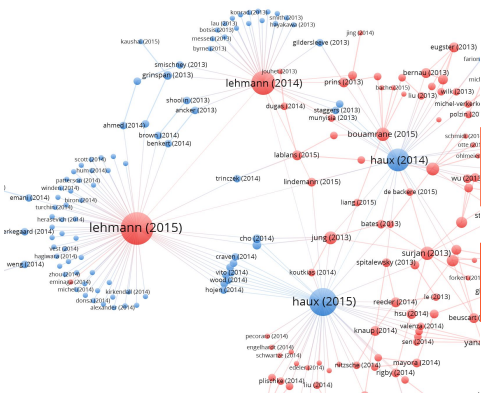
Why Graphs?

Why do we need GNNs?

Traditional deep learning models are great at handling **grids** (like images) and **sequences** (like text), but many real-world datasets don't fit neatly into these structures. Instead, they exist as **graphs**, where entities (nodes) are connected by relationships (edges)



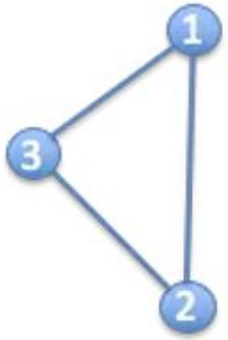




What is a Graph?

A graph is a data structure consisting of two components: **vertices**, and **edges**. It is used as a mathematical structure to analyze the pair-wise relationship between objects and entities. Typically, a graph is defined as **G=(V, E)**, where

- V is a set of nodes (e.g., cities, people)
- E is the edges between them (links or connections

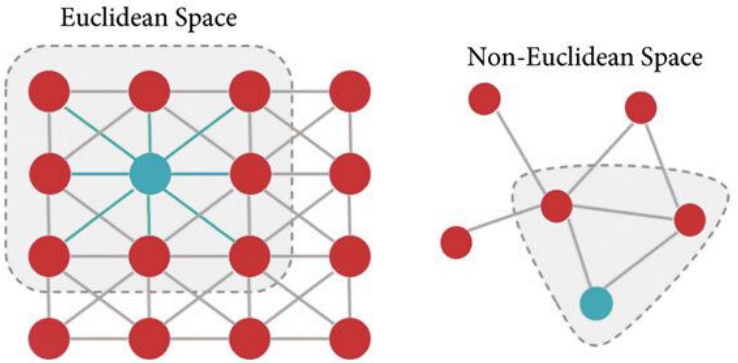


$G = (V, E)$

Non-Euclidean = no “up” or “down”

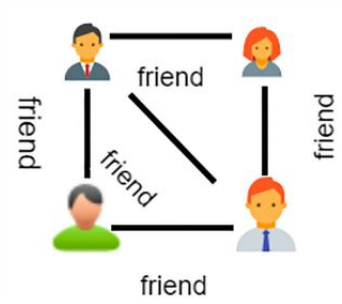
Non Euclidean

A graph does not exist in a Euclidean space, which means it cannot be represented by any coordinate systems that we are familiar with. This makes the interpretation of graph data much harder as compared to other types of data such as waves, images, or time-series signals("text" can also be treated as time-series), which can be easily mapped to a 2-D or 3-D Euclidean space.

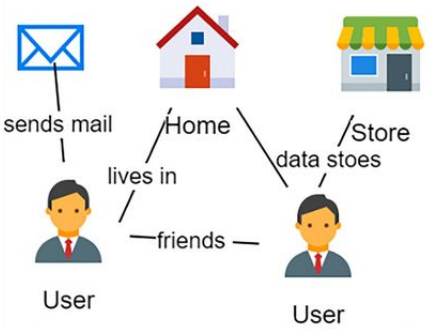


Types of Graphs?

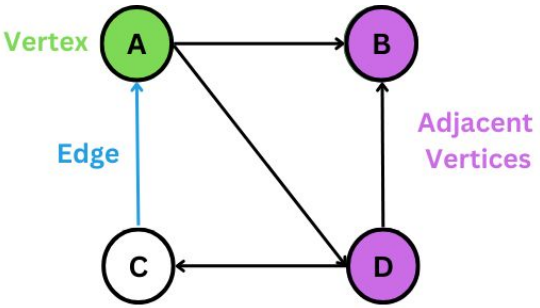
- Directed vs. Undirected
- Weighted vs. Unweighted
- Homogeneous vs. Heterogeneous



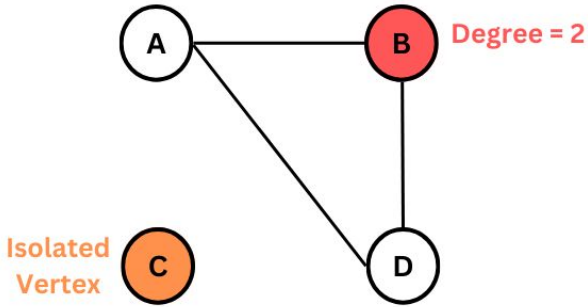
Homogeneous Graph



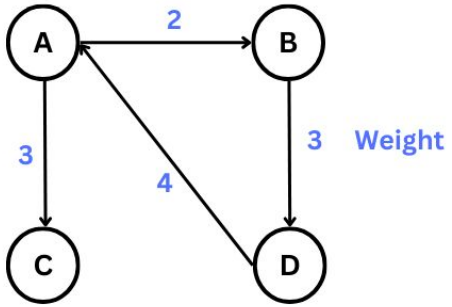
Heterogeneous Graph



Directed Graph



Undirected Graph

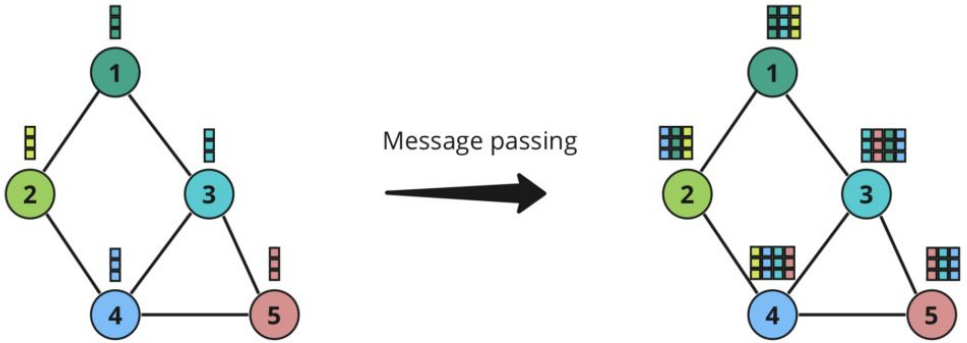


Weighted Graph

Traditional Deep Learning vs. Graph-Based Learning

- Traditional Deep Learning Models: **CNNs** process grid-like data (e.g., images). **RNNs** process sequential data (e.g., text, time series).
- **Graph Neural Networks:** Capture node relationships and global structures.

GNNs use message passing between connected nodes.



How GNNs work?

Message Passing

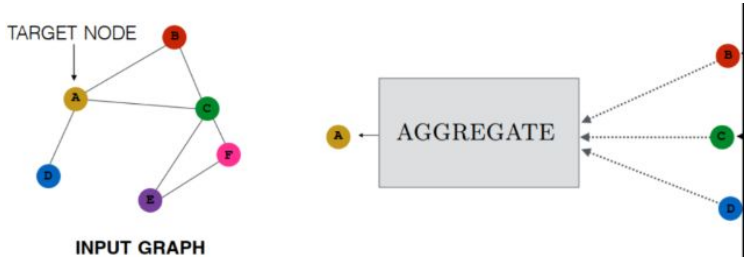
- **Core Idea:** Nodes learn by aggregating information from their neighbors.
- **Steps**: Feature Propagation: Nodes gather information from neighbors.
- **Aggregation**: Compute new node representations. Update: Update node embeddings iteratively.

Message Passing

$$\mathbf{h}_u^{(k+1)} = \text{UPDATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{AGGREGATE}^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\})\right)$$

Node-by-Node

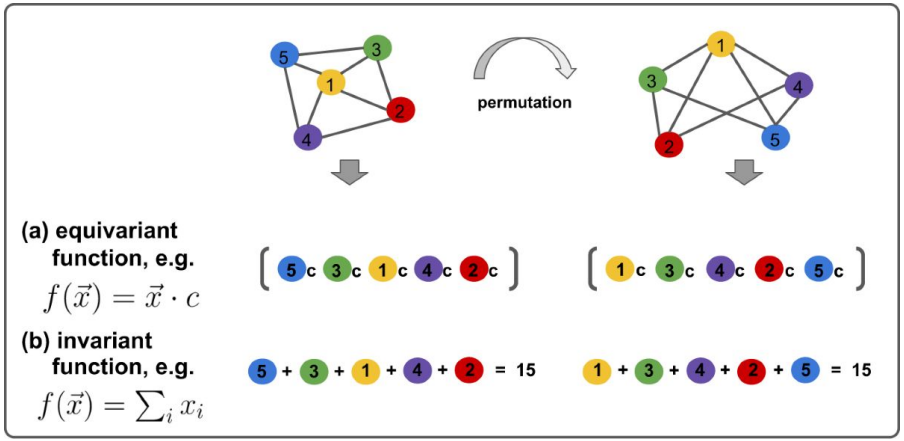
- *h* = node features / embeddings
- *k* = number of iterations



Aggregate function operates over sets, must be permutation invariant or permutation equivariant.

Permutation Equivariance

Since the order of neighbors doesn't matter, the function must work the same way no matter how the neighbors are arranged **(permutation-invariant)** or change predictably if the order changes **(permutation-equivariant).**



Message Passing

$$\mathbf{h}_u^{(k+1)} = \text{UPDATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{AGGREGATE}^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\})\right)$$

Neural message passing uses neural networks for the update and aggregate functions

$$h_u^{(k+1)} = \sigma\left(W_{\text{self}}^{(k+1)} h_u^k + W_{\text{neigh}}^{(k+1)} \sum_{v \in \mathcal{N}(u)} h_v^{(k)}\right)$$

Each node's updated value becomes a weighting of its previous value + a weighting of its neighbor's values.

The choice to sum over neighboring nodes isn't the only valid choice, other choices include mean, max, concatenation, etc.

Message Passing

$$h_u^{(k+1)} = \sigma\left(W_{\text{self}}^{(k+1)} h_u^k + W_{\text{neigh}}^{(k+1)} \sum_{v \in \mathcal{N}(u)} h_v^{(k)}\right)$$

$$\mathbf{H}^{(k+1)} = \sigma\left((\mathbf{A} + \mathbf{I})\mathbf{H}^{(k)}\mathbf{W}^{(k+1)}\right)$$

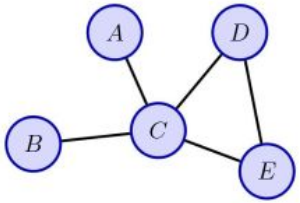
Neural message passing can be simplified to be efficient and generalizable by sharing parameters.

Collapse W_{self} and W_{neigh} into W by adding self-loops to the adjacency matrix A.

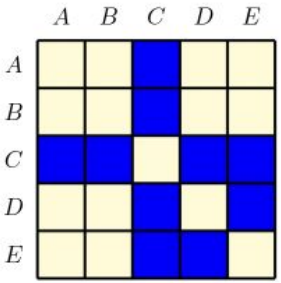
This method reduces message passing to relatively simple matrix multiplication

Adjacency Matrix

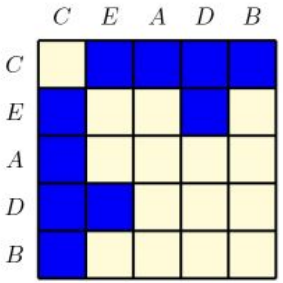
A graph is often represented by an Adjacency matrix, A.
If a graph has N nodes, then A has a dimension of (NxN).
People sometimes provide another feature matrix to describe the nodes in the graph.
If each node has F numbers of features, then the feature matrix X has a dimension of (NxF).



(a)



(b)



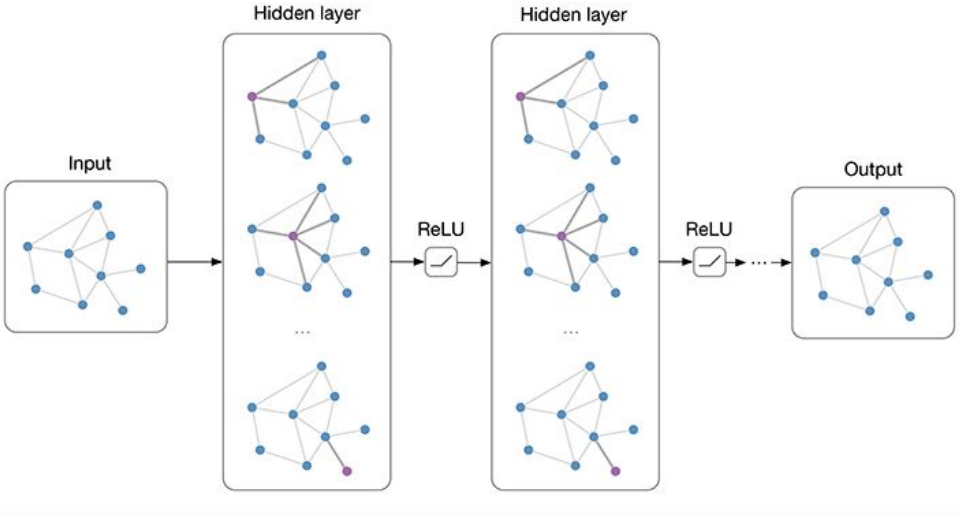
(c)

An example of an adjacency matrix showing (a) an example of a graph with five nodes, (b) the associated adjacency matrix for a particular choice of node order, and (c) the adjacency matrix corresponding to a different choice for the node order.

Types of GNNs

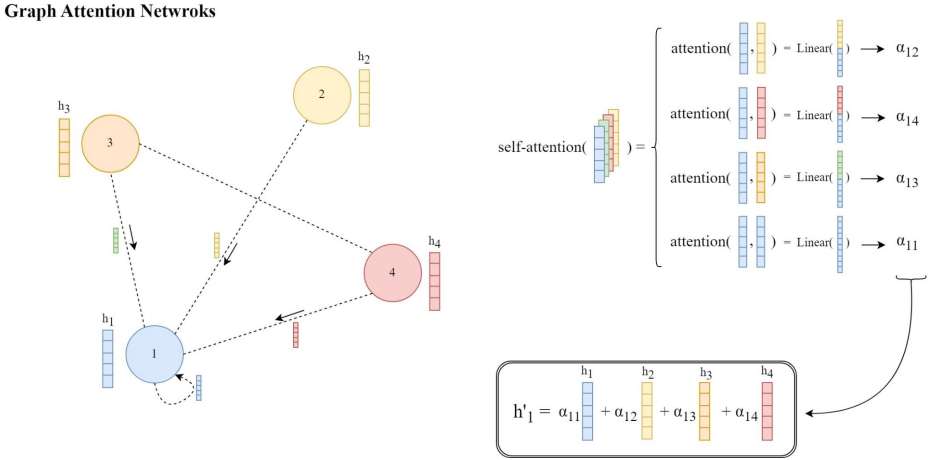
Extensions - Graph Convolutional Networks

- Just as convolutions in CNNs process pixel neighborhoods in an image, GCNs aggregate information from neighboring nodes to update their own features.
- Google has actively contributed to graph-based learning through Google Brain, DeepMind, and TensorFlow GNN.



Extensions - Graph Attention Networks

- Introduce the attention mechanism into Graph Neural Networks (GNNs) to improve how information is aggregated from neighbors.
- Traditional Graph Convolutional Networks (GCNs) aggregate information uniformly from all neighboring nodes. This can be problematic because: Not all neighbors are equally important
- GATs solve this by assigning different attention scores to different neighbors, allowing the network to focus on the most relevant connections.



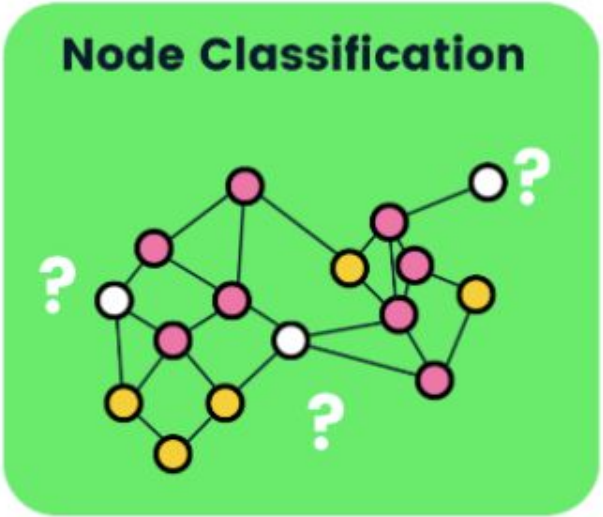
Extensions!

- Graph Transformers
- Temporal Graph Networks
- And more!

Applications

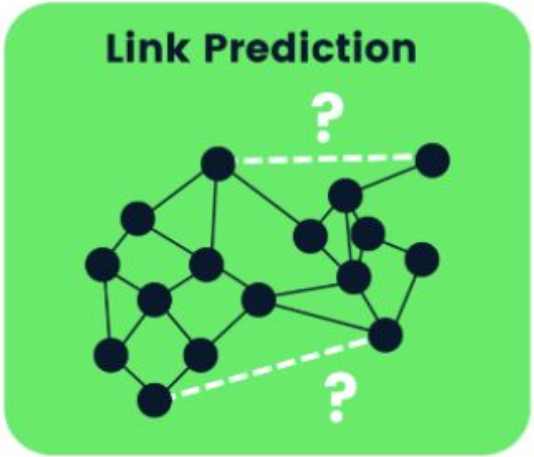
Tasks: Node Classification

- The task is to predict the node embedding for every node in a graph.
- This type of problem is usually trained in a semi-supervised way, where only part of the graph is labeled.
- Typical applications for node classification include citation networks, Reddit posts, Youtube videos, and Facebook friends relationships.



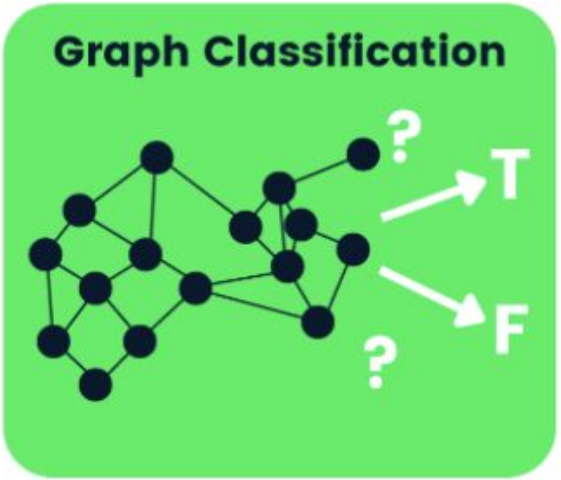
Tasks: Link prediction

- In link prediction, the task is to understand the relationship between entities in graphs and predict if two entities have a connection in between.
- For example, a recommender system can be treated as link prediction problem where the model is given a set of users' reviews of different products, the task is to predict the users' preferences and tune the recommender system to push more relevant products according to users' interest.

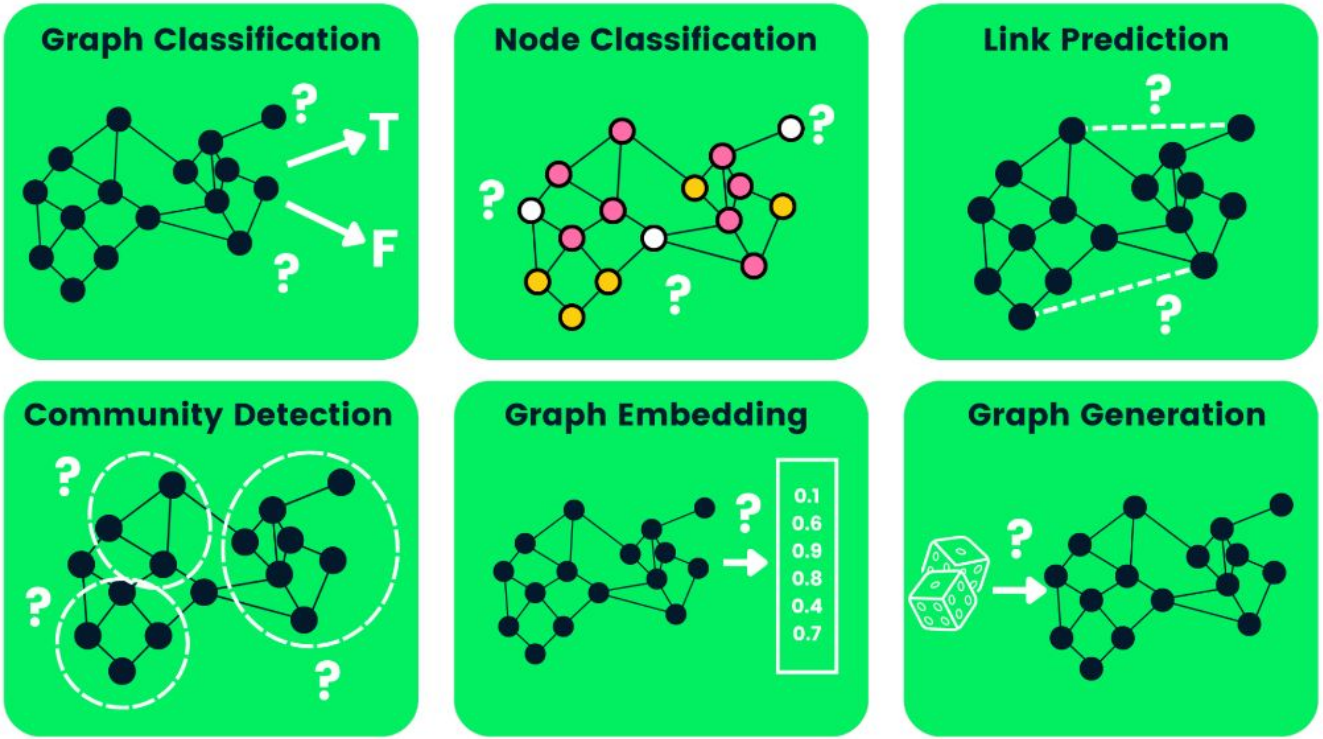


Tasks: Graph Classification

- The task is to classify the whole graph into different categories. It is similar to image classification but the target changes into graph domain.
- There is a wide range of industrial problems where graph classification can be applied, for example, in chemistry, biomedical, physics, where the model is given a molecular structure and asked to classify the target into meaningful categories.



And more!


Graph Classification
?
T
F
?
Node Classification
?
?
?
Link Prediction
?
?
Community Detection
?
?
?
Graph Embedding
?
0.1
0.6
0.9
0.8
0.4
0.7
Graph Generation
?

Implementation

Implementation

```python
import torch  # Import PyTorch for tensor operations

# Step 1: Define the graph using an adjacency matrix
A = torch.tensor([[0, 1, 1],  # Node 0 connected to Nodes 1 and 2
                  [1, 0, 1],  # Node 1 connected to Nodes 0 and 2
                  [1, 1, 0]], # Node 2 connected to Nodes 0 and 1
                 dtype=torch.float)

# Step 2: Define node features (each node has 2 features)
X = torch.tensor([[1.0, 2.0],  # Node 0 features
                  [2.0, 3.0],  # Node 1 features
                  [3.0, 1.0]], # Node 2 features
                 dtype=torch.float)

# Step 3: Add self-loops to the adjacency matrix
# -----------------------------------------------
# Why?
# - Ensures that each node retains some of its own features during message passing.
# - Prevents feature dilution when aggregating information from neighbors.
I = torch.eye(A.shape[0])  # Identity matrix (for self-loops)
A_hat = A + I  # A_hat is the adjacency matrix with self-loops
print("Adjacency Matrix (with self-loops):")
print(A_hat)

# Step 4: Compute the degree matrix (D)
# -------------------------------------------
# Why?
# - The degree matrix ensures that nodes with more connections don't overpower those with fewer connections.
# - Used to normalize adjacency matrix so that information spreads evenly.
D = torch.diag(A_hat.sum(dim=1))  # Diagonal matrix with row sums of A_hat
print("\nDegree Matrix:")
print(D)
```

```
Adjacency Matrix (with self-loops):
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]

Degree Matrix:
[[3. 0. 0.]
 [0. 3. 0.]
 [0. 0. 3.]]
```

Implementation

```
# Step 5: Normalize the adjacency matrix
# ----------------------------------------------
# Why?
# - Raw adjacency matrix sums messages from neighbors, but nodes with high degrees can dominate.
# - Normalization ensures that information from all nodes is treated fairly.
D_inv = torch.inverse(D)       # Compute inverse of degree matrix D
A_norm = torch.mm(D_inv, A_hat) # Compute normalized adjacency matrix (D^-1 * A_hat)
print("\nNormalized Adjacency Matrix:")
print(A_norm)

# Step 6: Define trainable weight matrix W (learnable parameters)
W = torch.randn((2, 2), requires_grad=True)  # Randomly initialize a 2x2 matrix

# Step 7: Define the forward pass of the GNN
def forward(X, A_norm, W):
    H = torch.mm(A_norm, X)  # Step 7.1: Aggregate features from neighbors (A_norm * X)
    H = torch.mm(H, W)       # Step 7.2: Apply a learnable transformation (H * W)
    return H

# Step 8: Apply GNN forward function
output = forward(X, A_norm, W)

# Step 9: Print original and transformed features
print("\nOriginal Node Features:")
print(X)

print("\nAggregated Node Features After GNN:")
print(output)
```

```
Normalized Adjacency Matrix:
[[0.3333 0.3333 0.3333]
 [0.3333 0.3333 0.3333]
 [0.3333 0.3333 0.3333]]

Original Node Features:
[[1. 2.]
 [2. 3.]
 [3. 1.]]

Aggregated Node Features After GNN:
[[-0.1288 -0.6853]
 [-0.1288 -0.6853]
 [-0.1288 -0.6853]]
```

Further Reading

Researcher	Contributions
Franco Scarselli	Introduced the first Graph Neural Network model (GNN) in 2009
Thomas Kipf	Developed Graph Convolutional Networks (GCNs) (2016)
Petar Veličković	Created Graph Attention Networks (GATs) (2017)
William L. Hamilton	Developed GraphSAGE for inductive learning on large graphs (2017)
Yoshua Bengio	Pioneered work on message-passing neural networks (MPNNs)
Michael Bronstein	Leading researcher in Geometric Deep Learning
Jure Leskovec	Developed DeepWalk, Node2Vec, and GraphSAGE for node embedding
Xavier Bresson	Worked on spectral GNNs and deep learning on graphs
Justin Gilmer	Contributed to Graph Neural Networks for chemistry
Zhitao Ying	Developed GNNs for molecular property prediction (Graphormer)